

Re: Historical APR Void / Replay — Review of your proposal

To: Rick | **From:** Jose | **Date:** 2026-09-01
Subject: Historical APR Void / Replay — Answers to your 5 questions
Ref: ~/wmpplus/docs/apr-void-historico-propuesta-rick.md | Code: backend/server.js:3955-4621

FOR DISCUSSION ONLY — No code or database changes until mutual approval

Rick,

Thank you for the detailed proposal. I have reviewed it against the current schema (AssessmentPaymentRegister, AssessmentRegister, AssessmentPaymentSummary, ResidentMaster) and the live server code. This review is read-only — no changes have been made.

Overall assessment: the architecture is sound and I approve the direction, subject to the four amendments and four findings detailed below.

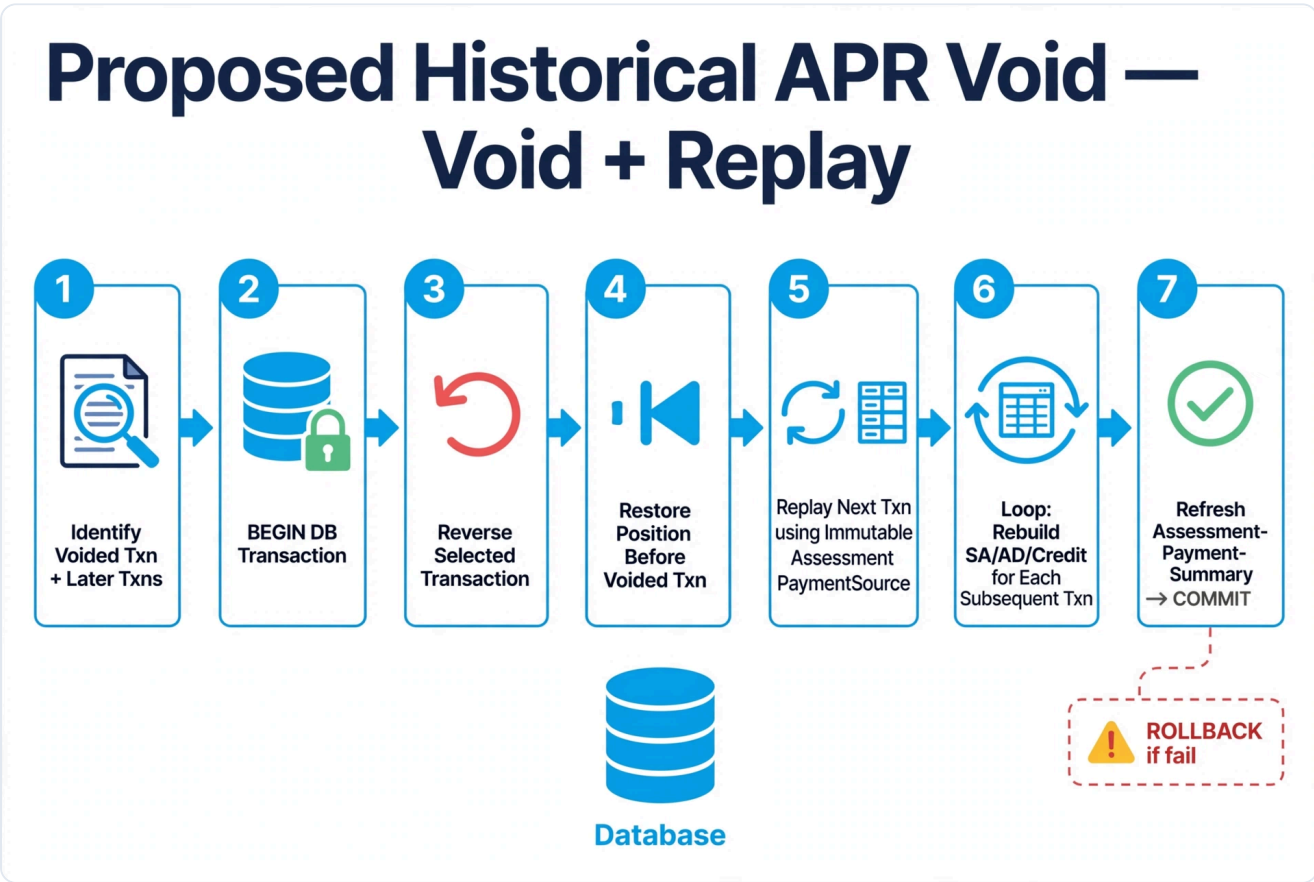


Figure 1 — Proposed Void + Replay flow (7 steps, atomic transaction). File: apr-void-replay_20260901-1830.png — meta/muse-image \$0.01

1. Answers to your five questions

Q1 — Is `AssessmentPaymentSource` the best way to preserve original intent? **YES.**

I tested whether intent could be derived from `AssessmentPaymentRegister` alone. It mostly can, but one case is ambiguous: an SA-field entry made when SA is already fully paid produces a single credit-only row with `PaymentType='AnnualDues'` — indistinguishable from a genuine Annual Dues entry. A Source table is therefore necessary.

Amendments: Add `GLNumber` and a `SubmissionKey` grouping the up to two transaction numbers one UF submission can create. Today `CashFlow` is keyed to only the first `TransactionNumber` (`server.js:4388-4417`), so voiding one of two txns from the same submission leaves the cash ledger ambiguous. Use `UNIQUE(TransactionNumber)`, one Source row per business transaction — consistent with your rule that a separately-entered Annual Dues amount keeps its own transaction number. Source never needs superseding — only `Status='VOID'`.

Q2 — Supersede and replace vs. edit in place? **SUPERSEDE AND REPLACE — agreed.**

Your example is definitive: a later \$100 that was *\$100 Resident Credit* can become *\$100 Special Assessment* after an earlier void. In-place editing cannot handle a change in row count or type.

Note: Use `Status` values `POSTED` / `SUPERSEDED` / `VOID` and keep `DeletedFlag='Y'` as “hidden.” Several queries filter `WHERE TransactionNumber=? AND DeletedFlag!='Y'` (e.g., `server.js:4576`). After replay a transaction number will have both superseded and active rows — those filters must become `Status='POSTED'`.

Q3 — Is `APRRecalculationBatchID` appropriate? **YES, with a leaner shape.**

I recommend a header table `APRRecalculationBatch` (`batch_id`, `resident`, `voided_transaction`, `operator`, `timestamp`, `reason`) plus two nullable columns on `AssessmentPaymentRegister`: `RecalcBatchID` and `ReplacesAPRRowID` for row-level lineage. That answers your audit question — “*Transaction X was voided, which required A, B, C, D to be replayed; these were the original allocations and these are the recalculated ones*” — without duplicating data.

Q4 — Rebuild from “immediately before the Void” or another baseline?

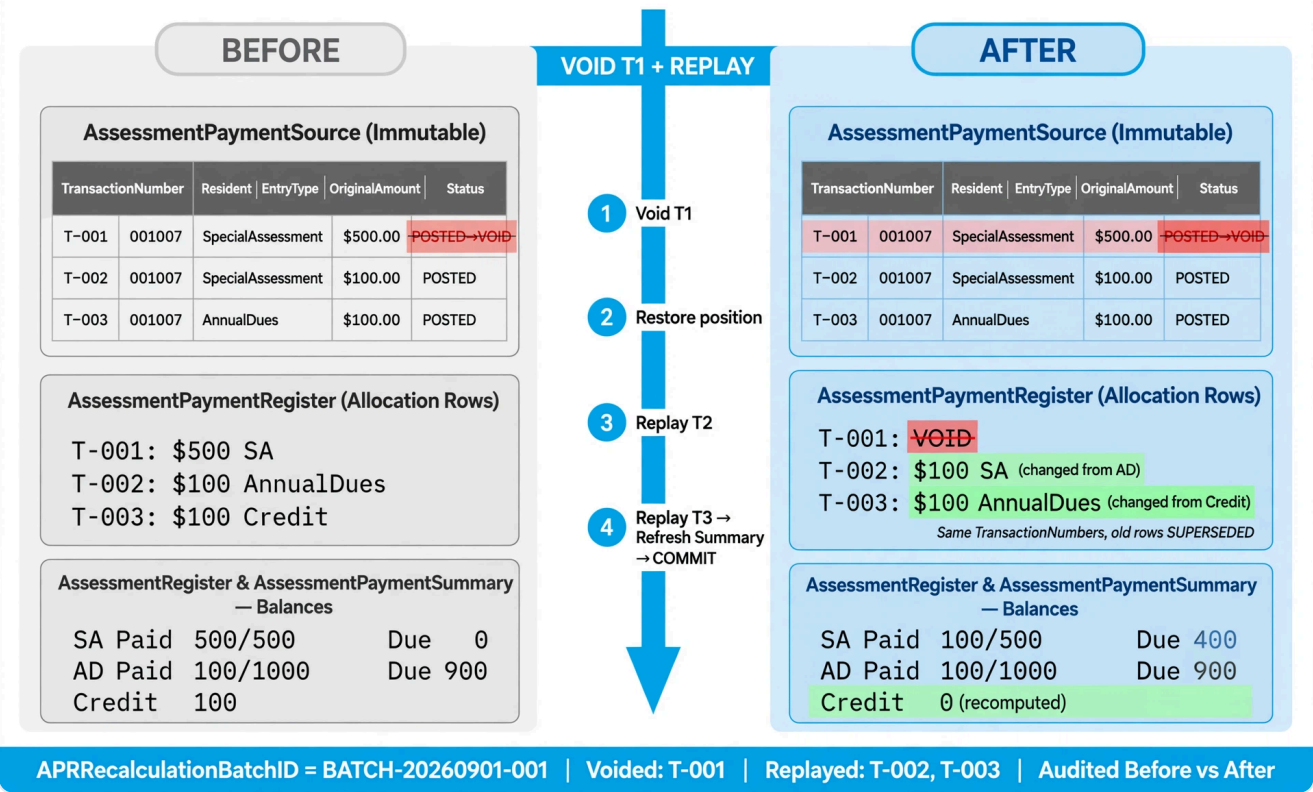
Your baseline is workable; I recommend a stronger invariant. Positions are pure functions of (charges, active APR rows). The current void rewinds aggregates subtractively with `GREATEST(...,0)` clamps (`server.js:4583-4602`), which can silently mask errors.

Recommendation: Do not rewind aggregates subtractively. After replay, **recompute** all aggregates from active APR rows. For the allocation replay, I recommend **replaying the full fiscal-year history from the start** — cost is trivial and removes dependence on baseline integrity.

Q5 — Database integrity / concurrency? Manageable with ordered locking.

Lock order: AssessmentRegister rows FOR UPDATE → all resident's APR rows → ResidentMaster row. Add GET_LOCK('apr-replay:<ResidentAccountID>') as second guard. Replay must use each Source record's stored CurrentFiscalYearBegins / PaymentDate , never re-derive from today. Dedupe on BatchID for idempotency. AssessmentPaymentSummary refreshed only once at the end — agreed.

APR Historical Void Simulation — Before vs After Replay (Sample Data)



F2	<code>enter-payment</code> never touches <code>AssessmentRegisterPeriod</code> , but void decrements <code>PeriodAmount</code>	<code>server.js:4363-4364</code> vs <code>4589-4592</code>	Void corrupts schedule data. Historical path should leave <code>Period</code> untouched.
F3	<code>ResidentMaster.ResidentCreditBalance</code> incremented on entry but never reversed on void	<code>server.js:4351-4361</code>	Both latest-void and historical replay must include it in rebuild.
F4	CashFlow is one row per UF submission, keyed to first <code>TransactionNumber</code> only	<code>server.js:4388-4417</code>	Voiding one of two txns from same submission voids CashFlow whose amount includes the other live txn's cash. Define rule; replay must never re-post CashFlow.

3. Open questions for your confirmation

1. Confirm that **F1–F4 fixes** should be folded into this procedure's scope.
2. **Full-history replay vs. minimal replay** — do you prefer the minimal replay you proposed or the full-year recomputation?
3. Is **Resident Credit** currently consumed by fines/AR (`CreditUsedForDuesAndViolationsYTD` is 0 today)? If so, replay must preserve consumption events.
4. Confirm the **CashFlow rule for shared-submission voids** (F4) and the simulation detail \$100 vs \$200 SA in Figure 2.

4. Proposed next steps

1. You confirm Q1–Q5 amendments and the four open questions above.
2. We backfill `AssessmentPaymentSource` for existing history (17 rows in dev) before go-live.
3. We land the void-primitive fixes (F1–F4), then the historical replay, all inside one DB transaction per your steps 4–12, with `AssessmentPaymentSummary` refresh only on success.

Again, **no server, React, or database changes will be made until we have mutual written approval** of the complete procedure.

Thank you for the thorough proposal — happy to walk through Figures 1 and 2 together.

Best regards,

Jose

Attachments: `apr-void-historico-propuesta-rick.md` (summary), `apr-void-replay_20260901-1830.png` (flow 193KB), `apr-void-simulacion_20260901.png` (simulation 314KB, Before/After with sample data — both meta/muse-image \$0.01) — both also in `~/wmpplus/public/workflows/`. Generations logged in `~/wmpplus/docs/workflows/generations.jsonl`.

Generated 2026-09-01 — For Discussion Only — 2 images inserted.